

OS Assignment 1

*) Operating System

→ Operating system is a system that interacts with software & hardware.

1) Batch operating system

→ A Batch operating system is a type of operating system in which multiple jobs are grouped into batches & processed sequentially without requiring direct interaction from the user during execution.

- Uses :-
 - 1> Payroll Processing
 - 2> Bank transaction Processing
 - 3> Exam result processing
 - 4> Electricity water & telephone bill
 - 5> large scale data processing.

- Advantages :-
 - 1> Efficient processing
 - 2> Reduces manual work
 - 3> Better CPU utilization
 - 4> Can process multiple jobs
 - 5> Reduces setup time.

- Disadvantages :-
 - 1> No direct user interaction
 - 2> long waiting time
 - 3> Difficult to debug
 - 4> Job priority issues
 - 5> Failure affects all jobs.

2) Multiprogramming operating System

→ A multiprogramming operating system is a type of OS in which multiple programs/jobs are kept in memory at the time. The CPU switches between them so that when one job is

waiting for I/O, the CPU can execute another job.

- Uses :-
 - 1) Interfaces CPU utilization by keeping the CPU busy
 - 2) Allows multiple programs to run at same time
 - 3) Improve overall system performance
- Advantages :-
 - 1) Better CPU utilization
 - 2) Reduces CPU idle time
 - 3) Increases system throughput
- Disadvantages :-
 - 1) operating system is complex to design
 - 2) Requires more main memory (RAM)

3) Time sharing operating system.

→ A Time sharing operating system (TSOS) is an operating system that allows multiple users to use the same computer at the same time by giving each user a small amount of CPU time is called a time slice.

- Uses :-
 - 1) Used in multi-user computer systems
 - 2) Allows many users to work simultaneously
- Advantages :-
 - 1) Supports multiple users at the same time
 - 2) Provides quick response to user
 - 3) Reduces CPU idle time
- Disadvantages :-
 - 1) Security & privacy issue may arise
 - 2) Failure of the central system affect all users

4) Multiprocessor operating system

→ A multiprocessor operating system is an OS that uses two or more CPUs (processors) in a single computer system.

- Uses :-
 - 1) Scientific research
 - 2) Database server
 - 3) Large business application
- Advantages :-
 - 1) Faster processing speed
 - 2) Better system performance
 - 3) Efficient resource sharing
- Disadvantages :-
 - 1) Complex operating system design
 - 2) Requires specialized
 - 3) Higher power consumption.

5) Real-Time operating system

→ A Real time operating system is a operations system that process data & responds to events within a fixed & guaranteed time limit. It is used where quick & predicate responses are essential.

- Uses :-
 - 1) Air traffic control system
 - 2) Medical equipment
 - 3) Embedded system
- Advantages :-
 - 1) Provides very fast response time
 - 2) Highly reliable & predictable
 - 3) Efficient use of system resources.
- Disadvantages :-
 - 1) Complex operating system design & programming
 - 2) Requires specialized hardware in some system.

⑥ Distributed operating system

→ A distributed OS is an OS that manages a group of independent computers & makes them work together as a single system. It allows users to share resources & perform tasks across multiple computers.

- Uses :- 1) cloud computing
2) large business networks
3) Banking systems
- Advantages :- 1) Efficient resources sharing
2) Easy to expand by adding more computers
- Disadvantages :- 1) complex to design & manage
2) Security & data privacy issues.

7) Network operating system

→ A Network operating system is an OS that manages computers & devices connected through a network. It allows users to share file printers, application & other resources over the network.

- Uses :- 1) File Sharing
2) Printer sharing
3) User account management
- Advantages :- 1) Easy sharing of files & resources
2) Improved Security through user authentication
3) Easy backup & recovery of data.
- Disadvantages :- 1) Requires a dedicated server.
2) Network failure can interrupt services.

8. Types of system calls

1. Process Control
→ Operations for starting, stopping, executing & managing processes as well as handling signals & process attributes.
2. File management
→ Operations for creating, opening, reading, writing, closing & manipulating files & their properties.
3. Device Management
→ Operations for requesting, reading from writing to & releasing hardware devices.
4. Information maintenance.
→ Operations for reading or setting system metadata, time data, process context & configuration.
5. Communication
→ Operation for creating connections, passing messages or establishing shared memory between processes.
6. Protection / Security.
→ Operations for controlling access permissions use IDs, group IDs, & resource limits.

9. System calls :-

- 1) Fork: Creates a new child process by duplicating the calling process.
- 2) vfork: creates a child process similar to fork, but suspends the parent process until the child either calls `execve` or `exit`. The child shares the address space with the parent temporarily to save overhead.
- 3) clone: A highly flexible linux-specific call used to create threads or new processes. It allows time-grained control over the resources shared between the parent & child.
- 4) Execve: loads & executes a new program inside the calling process's address space. It replaces the current process image, text, data, heap & stack with new binary.
- 5) Exit: Terminates the calling process gracefully flushing I/O buffers, registered cleanup functions & returning a status code to the parent.
- 6) _exit: Terminates the calling process immediately without performing high-level I/O buffer flushing or invoking registered cleanup handlers.
- 7) Wait: Blocks the calling parent process until any one of its child processes terminates returning the child's termination status.

- 8) `waitpid`: Suspends execution of the calling process until a specific child process changes State.
- 9) `waitid`: Provides more details control than `waitpid` over which child state changes to wait for & populates a detailed `siginfo_t` structure.
- 10) `Kill`: sends a specified signal to targeted process or process group.
- 11) `getpid`: Return the unique process ID of the calling process.
- 12) `getppid`: returns the process ID of the parent process that spawned the calling process.
- 13) `nice`: Adds a relative value to the calling process scheduling priority, affecting how much CPU time it gets relative to other processes.
- 14) `setpriority`: Sets the Scheduling priority of a target process group, or user ID.
- 15) `getpriority`: Retrieves the Scheduling priority of a target process, process group or user ID.
- 16) `Open`: opens a file at a specified Path, return

ning a file descriptor used for subsequent operations can also create the file if specified flags are set.

- 17) Create : Creates a new file or rewrites an existing file & opens it for writing.
- 18) close : closes an open file descriptor so that it no longer refers to any file & can be reused by the system.
- 19) Read : Reads a requested no. of bytes from an open file descriptor into a buffer.
- 20) Write : Writes a requested no. of bytes from a buffer out to an open file descriptor.
- 21) Pread : Reads bytes from a file descriptor at a specific file offset without updating the underlying offset pointer.
- 22) Pwrite : writes bytes to a file descriptor at a specific file offset without updating the underlying offset pointer.
- 23) lseek : Repositions the file offset pointer for a given file descriptor.
- 24) Stat : Retrieves details file metadata given a path to file.
- 25) lstat : Retrieves file metadata for a specified

path: unlike Stat, if the path points to a symbolic link, it returns information about the link itself rather than the target file.

26) fstat: Retrieves detailed metadata for a file using an open file descriptor instead of path.

27) Access: Checks whether the calling process has permission to read, write or execute a file at a specified path.

28) Unlike: deletes a name/link from the filesystem. It was last link to the file & no process has it open, the space is reclaimed.

29) Rename: changes the name or location of the file or directory on the filesystem.

30) mkdir: Creates a new, empty directory with specified access permissions.

31) rmdir: Deletes an empty directory from the filesystem.

32) dup: duplicates an open file descriptor, returning the lowest numbered available descriptor pointing to the same open file context.

33) dup2: Duplicates an open file descriptor, copying

its explicitly into a user specified descriptor number.

- 34) `fcntl`: Perform various control operations on an open file descriptor.
- 35) `fsync`: forces all modified in core data & metadata associated with a file descriptor to be written out to physical storage hardware.
- 36) `fdatasync`: similar to `fsync`, but syncs only a modified file data to physical disk skipping non-essential metadata updates like access time.
- 37) `truncate`: Shrinks or extends the length of a file at a specified path to a specified length.